CS 61A
Past Exam Practice — Generators and Counting

Your Name: _____

Your Student ID: _____

Room you are taking the exam in: _____

Name and SID of the person to your **left**: _____

Name and SID of the person to your **right**: _____

You have 110 minutes. There are 2 questions of varying credit. (0 points total)

Question	1	2	Total
Points	0	0	0

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◎ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Make sure to **write your student ID at the top of each page** in the provided blank.

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam. I will not use any other unauthorized materials or devices during this exam. I will not communicate with anyone other than the proctors during this exam. I will not share any information about the exam with anyone until after the end of the exam period.

Acknowledge that you have read and agree to the honor code above and **sign your name below**:

_____

This content is protected and may not be shared, uploaded, or distributed.

Q1 *Iterators Are Inevitable* **(0 points)**

Original source: Spring 2019 Final - Question 1 | **Level:** Regular

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. The first row is completed for you.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**.
- To display an iterator/generator value, write **Iterator**.
- If an expression would take forever to evaluate, write **Forever**.

The interactive interpreter displays the contents of the `repr` string of the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the code below first, then you evaluate each expression in the table in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
1  def love():
2      yield 1000
3      yield from [2000, 3000]
4
5  x = love()
6  L = list(x)
7
8  def alternate(real, ity):
9      i1, i2 = iter(real), iter(ity)
10     try:
11         while True:
12             yield next(i1)
13             yield next(i2)
14     except StopIteration:
15         yield 'inevitable'
16
17 thanos = ['power', 'space', 'reality']
18 tony = ['mind', 'soul', 'time']
19 i = iter(tony)
20 next(i)
21 tony.extend(list(i))
22 thanos = tony[2::-2]
```

Expression	**Output**
`pow(10, 2)`	`100`
`print(print('end', print('game')), x)`	
`L`	
`next(x)`	
`tony`	
`list(alternate(thanos[1:], thanos))`	

This content is protected and may not be shared, uploaded, or distributed.

Q2 *We're All the Same to Them* **(0 points)**

Original source: Summer 2017 Final - Question 1 | **Level:** Challenge

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If a function value is displayed, write "Function". If an iterator or generator value would be displayed, write "Iterator" (instead of something like `<iterator object at 0x...>`).

Recall: The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the following statements:

```
def tee(iterable):
    it = iter(iterable)
    queues = [[], []]
    def gen(lst):
        while True:
            if not lst:
                try:
                    value = next(it)
                except StopIteration:
                    return
                for q in queues:
                    q.append(value)
            yield lst.pop(0)
    return [gen(queues[0]), gen(queues[1])]

yum = ['avocado', 'quinoa', 'cream cheese']
```

Expressions are evaluated in the order shown and have a cumulative effect. The first row is completed for you.

Expression	**Output**
`print(yum[0] + ' ' + yum[-1]) * 5`	`avocado cream cheese` `Error`
`print(print(next(iter(yum))), next(yum))`	
`next(iter(next(iter(yum))))`	

`eat = iter(tee(yum))`
`neat = next(eat)`
`next(neat)`	

`yum[0] = yum.pop()`
`yum.append(next(neat))`
`yum[:2]`	

`next(neat)`	
`munch = next(iter(tee(yum)))` `next(munch) + ' ' + next(next(eat))`	

This content is protected and may not be shared, uploaded, or distributed.

(Question 2 continued…)

This content is protected and may not be shared, uploaded, or distributed.